

টেকনিক্যাল রিসার্চ গাইড

LoRA ও QLoRA

LLM Fine-Tuning-এর সম্পূর্ণ বাংলা গাইড

Low-Rank Adaptation

Quantized LoRA

Matrix Decomposition

Transformer Fine-Tuning

LLM Optimization

Transformer Architecture · Parameter Efficient Fine-Tuning · Bengali NLP Guide

ভূমিকা: LLM Fine-Tuning কেন দরকার?

আজকের দিনে AI জগতে GPT-4, LLaMA, Mistral, Gemini-এর মতো বিশাল Large Language Model (LLM) গুলো ইন্টারনেট, বই, কোড সহ কোটি কোটি টেক্সট থেকে প্রি-ট্রেন করা হয়। এই মডেলগুলো সাধারণ প্রশ্নের উত্তর দিতে পারে, কোড লিখতে পারে, গল্প লিখতে পারে। কিন্তু যখন কোনো নির্দিষ্ট কাজের জন্য — যেমন বাংলা চ্যাটবট, মেডিক্যাল Q&A, বা কাস্টমার সাপোর্ট — এই মডেলগুলো ব্যবহার করতে হয়, তখন সেগুলোকে নতুন ডাটা দিয়ে আরও শেখাতে হয়। এই প্রক্রিয়াকেই বলা হয় **Fine-Tuning**।

Fine-Tuning-এর বিভিন্ন ধরন

Fine-tuning মূলত তিনটি প্রধান ধরনে হয়ে থাকে। প্রথমত, **Full Parameter Fine-Tuning** — এখানে মডেলের সমস্ত ওজন (weight) আপডেট করা হয়। দ্বিতীয়ত, **Domain-Specific Fine-Tuning** — নির্দিষ্ট ডোমেইন যেমন Finance, Medical বা Legal-এর জন্য। তৃতীয়ত, **Task-Specific Fine-Tuning** — Text-to-SQL, Document Q&A, Summarization ইত্যাদি নির্দিষ্ট কাজের জন্য।

✗ Full Parameter Fine-Tuning-এর সমস্যা

- 175 বিলিয়ন প্যারামিটার আপডেট করতে হয়
- বিশাল GPU/RAM প্রয়োজন (100+ GB VRAM)
- প্রচুর সময় ও অর্থ খরচ হয়
- Downstream task-এ ব্যবহার করা কঠিন
- Catastrophic Forgetting-এর ঝুঁকি থাকে

✓ LoRA দিয়ে সমাধান

- মাত্র কয়েক মিলিয়ন প্যারামিটার ট্রেন করতে হয়
- সাধারণ consumer GPU-তেই চলে
- দ্রুত ও সাশ্রয়ী
- Multiple task-এর জন্য আলাদা adapter রাখা যায়
- মূল মডেলের জ্ঞান অক্ষুণ্ণ থাকে



মূল ধারণা

Full Fine-Tuning-এ যদি 175 বিলিয়ন প্যারামিটার আপডেট করতে হয়, সেটি একটি সাধারণ ব্যক্তির পক্ষে সম্পূর্ণ অসাধ্য। LoRA এই সমস্যার সমাধান করে মাত্র কয়েক লক্ষ প্যারামিটার ট্রেন করে একই ফলাফল অর্জন করে।

LLM-এর প্যারামিটার ও ম্যাট্রিক্সের গঠন

LoRA বোঝার আগে আমাদের জানতে হবে একটি LLM-এর ভেতরে প্যারামিটারগুলো কীভাবে সাজানো থাকে। কারণ LoRA ঠিক এই ম্যাট্রিক্স কাঠামোকেই কাজে লাগায়।

সব প্যারামিটারের Shape কি একই?

না, একটি LLM-এর সকল প্যারামিটারের Shape বা Size একই হয় না। একটি Transformer আর্কিটেকচারের মডেলে বিভিন্ন ধরনের প্যারামিটার থাকে এবং তাদের আকার নির্ভর করে নির্দিষ্ট Layer ও Matrix Operation-এর উপর।

প্রধান প্যারামিটার ধরনসমূহ

প্যারামিটার টাইপ	সাইজ (hidden=768)	কাজ
Query Matrix (W_Q)	768×768	ইনপুটকে Query-তে রূপান্তর
Key Matrix (W_K)	768×768	ইনপুটকে Key-তে রূপান্তর
Value Matrix (W_V)	768×768	ইনপুটকে Value-তে রূপান্তর
FFN Layer 1	768×3072	ইনপুটকে প্রসারিত করা
FFN Layer 2	3072×768	আবার সংকুচিত করা
LayerNorm (γ, β)	768	Normalization
Bias Vector	768	Offset যোগ করা

ম্যাট্রিক্সের কাজ বোঝার উদাহরণ

ধরো $\text{hidden_size} = 4$ (বোঝার সুবিধায় ছোট রেখেছি)। একটি Query Weight Matrix দেখতে হবে এরকম:

W_{original} (Query Weight) — সাইজ: 4×4

$W_{1\ 1}$	$W_{1\ 2}$	$W_{1\ 3}$	$W_{1\ 4}$
$W_{2\ 1}$	$W_{2\ 2}$	$W_{2\ 3}$	$W_{2\ 4}$
$W_{3\ 1}$	$W_{3\ 2}$	$W_{3\ 3}$	$W_{3\ 4}$
$W_{4\ 1}$	$W_{4\ 2}$	$W_{4\ 3}$	$W_{4\ 4}$

● নীল = Frozen (পরিবর্তন হবে না) | মোট প্যারামিটার: 16টি



বাস্তবে কত বড়?

বাস্তব মডেলে $\text{hidden_size} = 4096$ বা 8192 হয়। একটি Query Matrix হবে $4096 \times 4096 = 16.7$ মিলিয়ন প্যারামিটার। শুধু একটি লেয়ারের একটি ম্যাট্রিক্সই! পুরো মডেলে শত শত এরকম ম্যাট্রিক্স আছে।

LoRA: Low-Rank Adaptation — মূল কার্যপদ্ধতি

LoRA কী?

LoRA (Low-Rank Adaptation of Large Language Models) হলো Microsoft-এর তৈরি একটি Parameter-Efficient Fine-Tuning পদ্ধতি। এটি মেইন মডেলের কোনো ওজন পরিবর্তন না করে, মডেলের পাশে কিছু ছোট ম্যাট্রিক্স যোগ করে নতুন জ্ঞান শেখায়।

Matrix Decomposition — মূল কৌশল

LoRA-এর হৃদয়ে আছে একটি গণিতের কৌশল — **Matrix Decomposition**। ঠিক যেভাবে আমরা সংখ্যাকে উৎপাদকে ভাঙি (যেমন $১২ = ৩ \times ৪$), তেমনি একটি বড় ম্যাট্রিক্সকে দুটি ছোট ম্যাট্রিক্সে ভাঙা যায়।

$$W_{new} = W_0 + \Delta W = W_0 + (B \times A)$$

W_0 = মূল Pre-trained Weight (Frozen) | $B \times A$ = LoRA-এর শেখানো নতুন পরিবর্তন
(Trainable)

Rank কী এবং কেন গুরুত্বপূর্ণ?

Rank (r) হলো একটি ছোট সংখ্যা যা LoRA ম্যাট্রিক্সের আকার নির্ধারণ করে। ধরো মূল ম্যাট্রিক্স W -এর সাইজ $d \times d$ । LoRA তখন দুটি ম্যাট্রিক্স তৈরি করে:

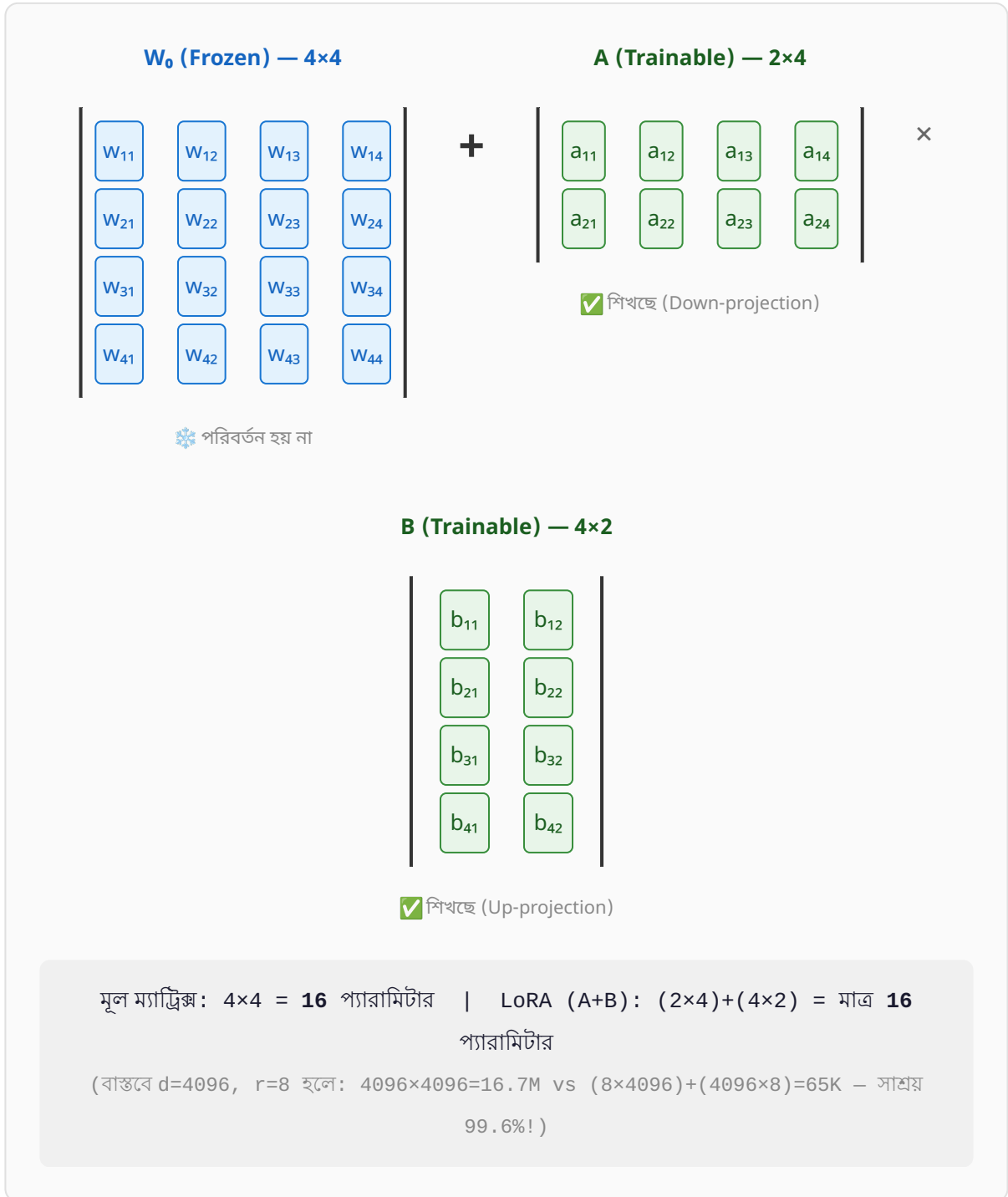
ম্যাট্রিক্স **A**: সাইজ = $r \times d$ (ইনপুটকে সংকুচিত করে)

ম্যাট্রিক্স **B**: সাইজ = $d \times r$ (আবার প্রসারিত করে)

যেখানে $r \ll d$ (র‍্যাঙ্ক মেইন ডাইমেনশনের চেয়ে অনেক ছোট)

ভিজ্যুয়াল উদাহরণ: $d=4$, Rank $r=2$

মনে করো $\text{hidden_size } (d) = 4$ এবং $\text{rank } (r) = 2$ । নিচে LoRA-এর পুরো কাঠামো দেখো:



প্যারামিটার সংখ্যার তুলনা

175B

4.7M

99.97%

প্যারামিটার সাশ্রয়

Full Fine-Tuning
প্যারামিটার

LoRA (Rank=4)
প্যারামিটার

Rank অনুযায়ী প্যারামিটার তুলনা

Model Size	Rank = 1	Rank = 4	Rank = 8	Rank = 64
7 Billion	167K	671K	1.34M	10.7M
13 Billion	228K	912K	1.82M	14.6M
70 Billion	529K	2.1M	4.2M	33.9M
180 Billion	849K	3.4M	6.8M	54.3M



কখন HIGH RANK ব্যবহার করবে?

যদি মডেলকে **জটিল বিষয়** শেখাতে হয় — যেমন এমন কাজ যা মডেল আগে কখনো দেখেনি — তখন High Rank (যেমন 32, 64) ব্যবহার করা উচিত। সাধারণ task-এর জন্য Rank 4 থেকে 16 যথেষ্ট। Microsoft-এর গবেষণায় Rank 8 সবচেয়ে ভালো performance দিয়েছে।

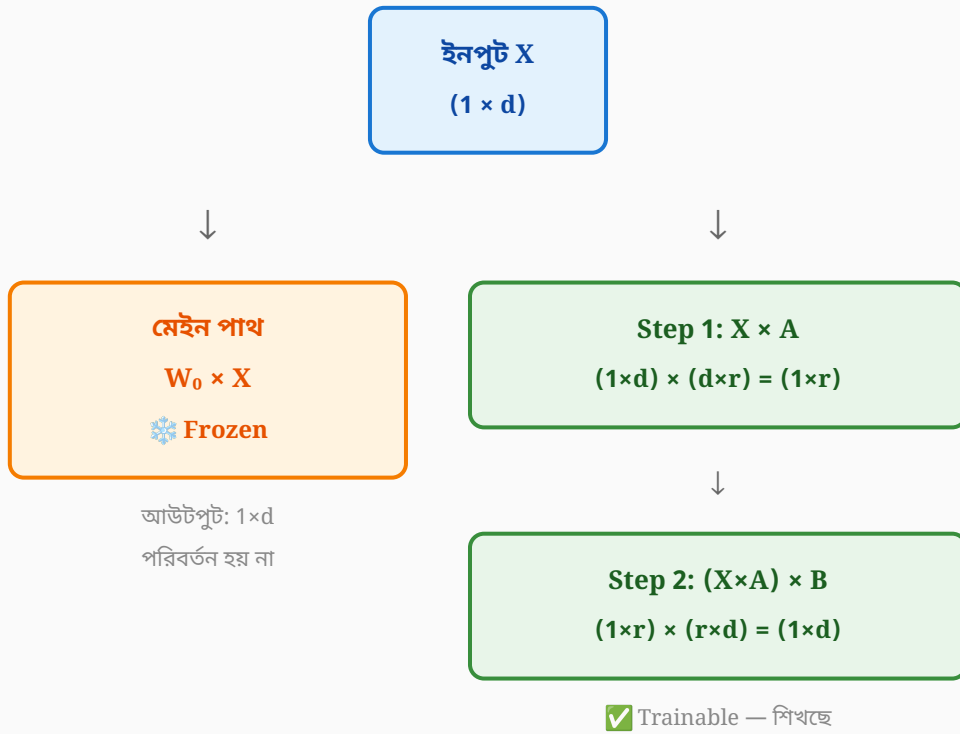
LoRA Training Process — ধাপে ধাপে শেখার পদ্ধতি

Training-এর সময় কী ঘটে?

LoRA Fine-Tuning একটি single pass-এ হয়, দুবার আলাদাভাবে নয়। যখন ডাটা মডেলে যায়, তখন মেইন মডেলের frozen weight এবং LoRA-এর trainable weight — উভয়ই একসাথে কাজ করে। কিন্তু Backpropagation-এ শুধু LoRA-এর A ও B ম্যাট্রিক্স আপডেট হয়।

LoRA Path — দুটি সমান্তরাল রাস্তা

Forward Pass: একটি ইনপুট, দুটি পথ



$$\text{আউটপুট} = W_0X + BAX$$

দুই পথ যোগ হয়

গাণিতিক ব্যাখ্যা — ধাপে ধাপে

ধরো ইনপুট X -এর সাইজ 1×4 (চারটি feature):

- মেইন পাথ:** $X (1 \times 4) \times W_0 (4 \times 4) = \text{আউটপুট} (1 \times 4)$
মূল প্রি-ট্রেন্ড জ্ঞান ব্যবহার হচ্ছে, কিন্তু কিছু পরিবর্তন হচ্ছে না।
- LoRA Down-Projection (A):** $X (1 \times 4) \times A (4 \times 2) = (1 \times 2)$
4 dimension থেকে মাত্র 2 dimension-এ সংকুচিত হলো। এই 2-ই হলো আমাদের rank।
- LoRA Up-Projection (B):** $(1 \times 2) \times B (2 \times 4) = (1 \times 4)$
আবার 4 dimension-এ ফিরে এলো। এখন মেইন পাথের সাথে যোগ করা সম্ভব।
- চূড়ান্ত আউটপুট:** $\text{Output} = (X \times W_0) + (X \times A \times B)$
দুই পথের ফলাফল যোগ হয়ে চূড়ান্ত আউটপুট তৈরি হয়।

$$h = W_0x + BAx = (W_0 + BA)x$$

h = চূড়ান্ত আউটপুট | W_0 = frozen weight | B, A = trainable LoRA matrices

Zero Initialization — LoRA-এর চমৎকার কৌশল



শূন্য দিয়ে শুরু

Training-এর শুরুতে ম্যাট্রিক্স B -কে সম্পূর্ণ শূন্য (0) দিয়ে initialize করা হয়। ফলে $B \times A = 0$, অর্থাৎ শুরুতে LoRA পাথের কোনো প্রভাব নেই। মডেল তার মূল প্রি-ট্রেন্ড জ্ঞান থেকে শুরু করে এবং ধীরে ধীরে নতুন জ্ঞান শিখতে থাকে।

কোন Matrix-এ LoRA প্রয়োগ করা হয়?

একটি LLM-এ শত শত ম্যাট্রিক্স আছে। LoRA সব ম্যাট্রিক্সে প্রয়োগ না করে শুধু নির্দিষ্ট কিছু লেয়ারে প্রয়োগ করা হয় — এগুলোকে `target_modules` বলে। গবেষণায় দেখা গেছে, Attention Layer-এর Q ও V ম্যাট্রিক্সে LoRA প্রয়োগ করলে সবচেয়ে ভালো ফলাফল পাওয়া যায়।

Module	প্রভাব	সাধারণত ব্যবহার?
q_proj (Query)	Attention Pattern পরিবর্তন	✓ হ্যাঁ
v_proj (Value)	কন্টেন্ট বোঝার ধরন পরিবর্তন	✓ হ্যাঁ
k_proj (Key)	কনটেক্সট বোঝার ধরন পরিবর্তন	কখনো কখনো
o_proj (Output)	Attention আউটপুট পরিবর্তন	কখনো কখনো
gate_proj / up_proj	FFN-এর জ্ঞান পরিবর্তন	Advanced ক্ষেত্রে

বাস্তব উপমা: ফ্যাক্টরি মডেল

একটি বিশাল ফ্যাক্টরি কল্পনা করো (মেইন মডেল)। প্রতিটি ঘরে কাঁচামাল (Input) প্রসেস হয়ে পরের ঘরে যায়। LoRA হলো বিশেষজ্ঞ কর্মচারী যারা নির্দিষ্ট কিছু ঘরে (Target Layers) বসানো হয়েছে। কাঁচামাল যখন ওই ঘরে পৌঁছায়, সাধারণ কর্মচারী (Frozen W_0) তার কাজ করে, আর বিশেষজ্ঞ (LoRA $A \times B$) তার নিজস্ব স্টাইলে নতুন কিছু যোগ করে দেয়। পুরো ফ্যাক্টরি সচল থাকে, কিন্তু আউটপুট বদলে যায়।

Inference (Prediction) — প্রেডিকশনের সময় কী হয়?

Training vs Inference পার্থক্য

Training-এর সময় ভাটা দুটি সমান্তরাল পথ দিয়ে যায়। কিন্তু Inference (প্রেডিকশন)-এর সময় একটি জাদুকরী প্রক্রিয়া ঘটে — **Weight Merging**।

Inference Process — Weight Merging

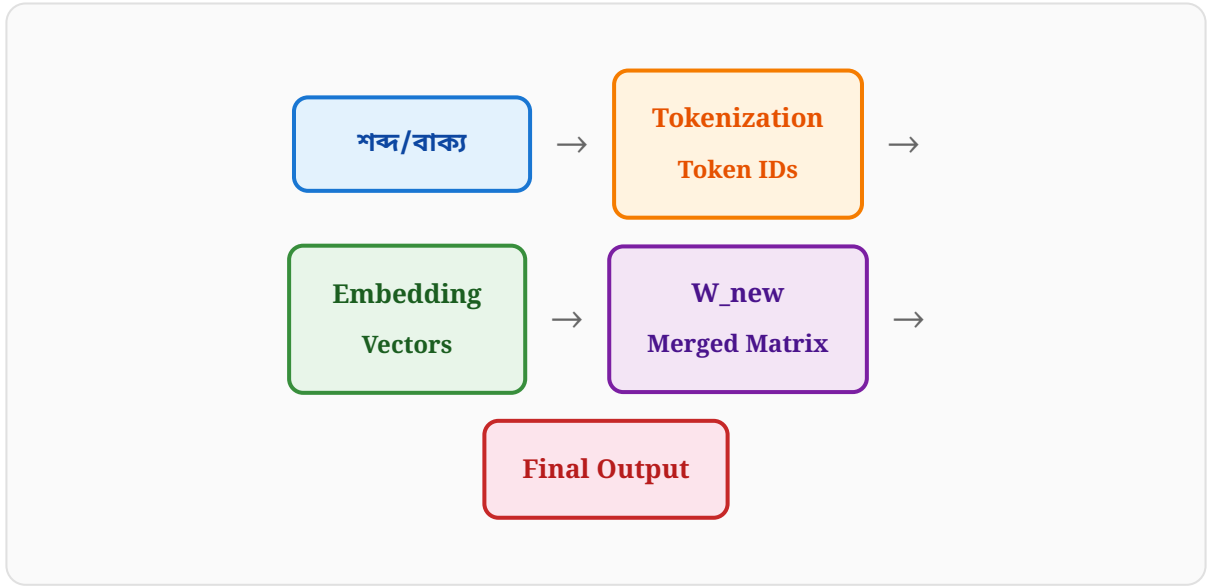
- 1 **Merge করো:** $W_{new} = W_0 + (B \times A)$ — মেইন ওজনের সাথে LoRA যোগ করো
- 2 **একক ম্যাট্রিক্স:** এখন শুধু একটাই ম্যাট্রিক্স W_{new} আছে, যেখানে দুটির জ্ঞান মিলে গেছে
- 3 **Input আসলে:** শব্দ $\rightarrow$ Token $\rightarrow$ Embedding Vector $\rightarrow$ W_{new} -এর মধ্য দিয়ে $\rightarrow$ Output

$$W_{new} = W_0 + BA$$

Inference-এর সময় এই একটিমাত্র merged matrix ব্যবহার হয়।

আলাদা দুটি পথ নেই — তাই কোনো extra latency নেই!

Inference-এর সম্পূর্ণ Flow



Adapter Swapping — একাধিক Task-এর সমাধান



ADAPTER পরিবর্তন করা

যদি একই মডেল দিয়ে দুটি ভিন্ন কাজ করতে হয় — যেমন একটি Medical chatbot এবং একটি Legal chatbot — তাহলে প্রতিটি task-এর জন্য আলাদা LoRA adapter (A ও B matrix) তৈরি করা হয়। Inference-এর সময় দ্রুত adapter পরিবর্তন করা যায়: বর্তমান LoRA বিয়োগ করে নতুন LoRA যোগ করো। এটিকেই "Adapter Swapping" বলা হয়।

Embedding — শব্দ থেকে Vector

Inference-এ যাওয়ার আগে, একটু জানা দরকার — Tokenization-এর পর শব্দগুলো কীভাবে Vector হয়।

- 1 **Token ID:** "Cat" → 5, "Dog" → 8, "Car" → 12 (ডিকশনারি দেখে)
- 2 **Embedding Lookup:** প্রতিটি ID-এর জন্য একটি লম্বা Vector টেবিল থেকে তুলে নেওয়া হয়
- 3 **Semantic Space:** Cat = [0.9, 0.8, 0.0], Dog = [0.9, 0.7, 0.0], Car = [0.0, 0.0, 0.9]
— অর্থের মিল থাকলে Vector কাছাকাছি থাকে

বিখ্যাত উদাহরণ: $\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$

Vector space-এ এই গণিত কাজ করে কারণ Embedding শব্দের অর্থ ধরে রাখে।

QLoRA — Quantized LoRA (LoRA 2.0)

QLoRA কী?

QLoRA হলো LoRA-এর উন্নত সংস্করণ, যেটাকে অনেকে **LoRA 2.0**ও বলে। এটি LoRA-এর উপরে Quantization নামের আরেকটি কৌশল প্রয়োগ করে। ফলে এটি আরও কম মেমরিতে আরও বড় মডেল Fine-Tune করতে পারে।

Quantization কী?

Quantization মানে হলো সংখ্যার Precision কমানো। সাধারণত মডেলের Weight গুলো Float 16-bit-এ থাকে। QLoRA এগুলোকে 4-bit-এ রূপান্তর করে।

16 FP16 / BF16 মূল Weight বেশি মেমরি	8 INT8 মধ্যবর্তী কম মেমরি	4 NF4 QLoRA ব্যবহার করে সর্বনিম্ন মেমরি
--	---	---

QLoRA-এর কাজের পদ্ধতি

- 1 Base Model Quantize:** Pre-trained মডেলের সব Weight-কে 4-bit NF4 format-এ রূপান্তর করা হয়।
- 2 LoRA Adapter তৈরি:** মূল কাঠামোতেই LoRA-এর A ও B ম্যাট্রিক্স যোগ করা হয়।
- 3 Double Quantization:** Quantization-এর Constants নিজেরাও আবার Quantize করা হয় — আরও মেমরি বাঁচায়।
- 4 Training:** Frozen 4-bit Weight রেখে শুধু LoRA ম্যাট্রিক্স 16-bit-এ ট্রেন হয়।

LoRA vs QLoRA তুলনা

বিষয়	LoRA	QLoRA
Base Model Precision	FP16 (16-bit)	NF4 (4-bit)
GPU RAM প্রয়োজন	বেশি	অনেক কম
65B মডেল ট্রেন করতে	~780GB VRAM	মাত্র ~48GB
Training Speed	দ্রুত	একটু ধীর
Accuracy	বেশি	প্রায় সমান
সাধারণ GPU-তে কাজ করে?	মাঝারি মডেলে হ্যাঁ	বড় মডেলেও হ্যাঁ



QLoRA-এর বিশাল অবদান

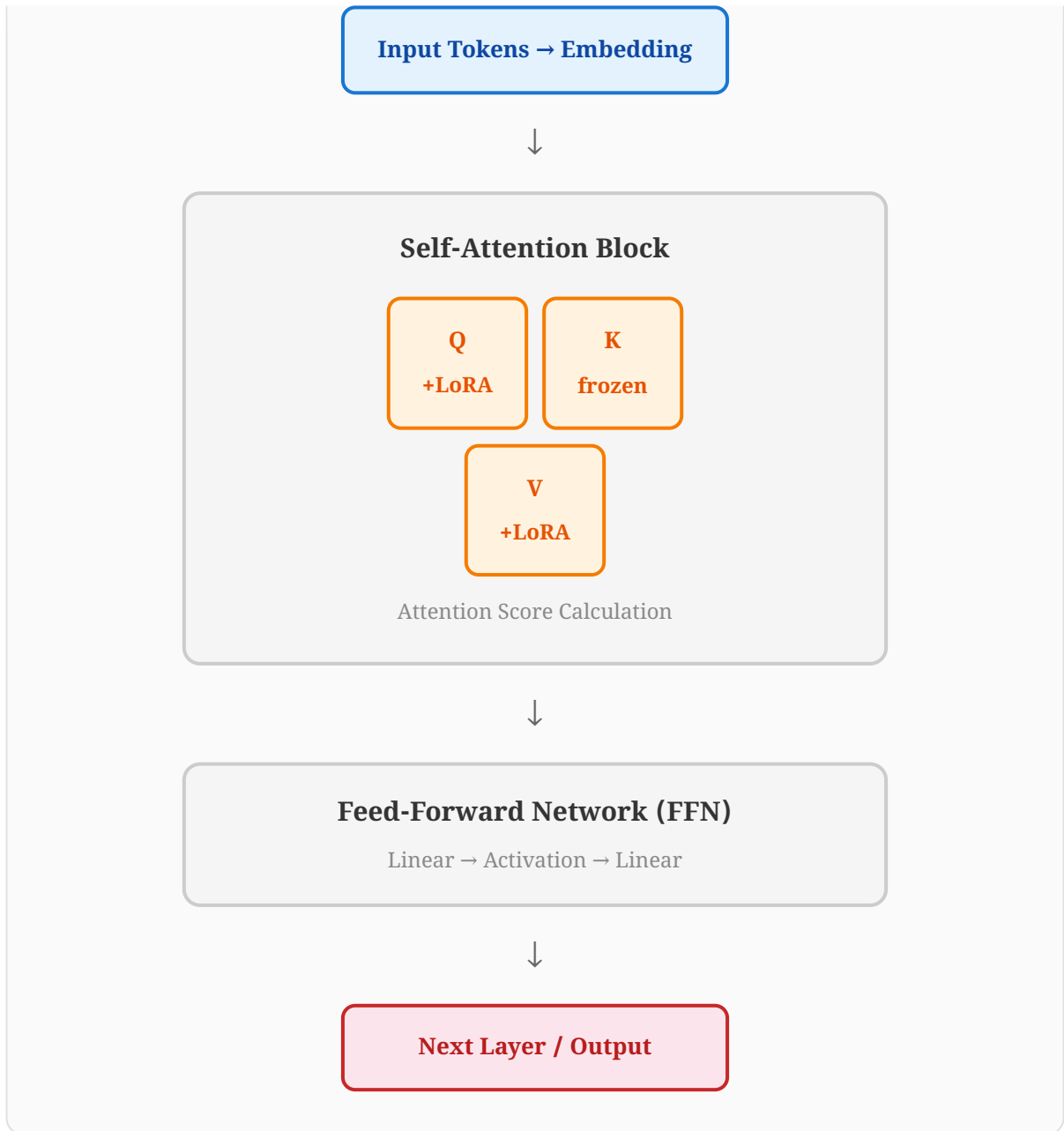
QLoRA ছাড়া 65 বিলিয়ন parameter-এর মডেল Fine-Tune করতে হলে প্রায় 780GB VRAM লাগত — যা একটি enterprise-grade server cluster। QLoRA দিয়ে মাত্র একটি A100 GPU (48GB VRAM)-এ এটি সম্ভব হয়েছে। এটি AI democratization-এর এক বিশাল মাইলফলক।

সম্পূর্ণ কাঠামো ও সংযোগ — Transformer এর সাথে সম্পর্ক

Transformer Architecture-এ LoRA কোথায় বসে?

একটি Transformer মডেলে প্রতিটি Layer-এ Self-Attention এবং Feed-Forward Network (FFN) থাকে। LoRA মূলত Self-Attention-এর Linear Projection Layer-গুলোতে বসানো হয়।

একটি Transformer Layer-এর ভেতরে LoRA-এর অবস্থান



কেন পুরো Architecture সক্রিয় থাকে?

LoRA Fine-Tuning-এ Input সবসময় পুরো মডেলের সব Layer-এর মধ্য দিয়ে যায়। মডেলের প্রথম দিকের Layer-গুলো ভাষা চেনার কাজ করে, মাঝের Layer-গুলো ব্যাকরণ ও যুক্তি বোঝে, আর শেষের Layer-গুলো নির্দিষ্ট কাজের উপযোগী ফলাফল তৈরি করে। শুধু LoRA Adapter-এ Input দিলে আগের Layer-এর আউটপুট পাওয়া যেত না — মডেল কিছুই বুঝতে পারত না।

সারসংক্ষেপ: পুরো মডেলটি Active থাকে, কিন্তু শুধু LoRA Adapter-এর A ও B ম্যাট্রিক্স Backpropagation-এ আপডেট হয়। বাকি সব Frozen থাকে।

Catastrophic Forgetting কীভাবে রোধ হয়?



মূল জ্ঞান অক্ষুণ্ণ থাকে

Full Fine-Tuning-এ সব Weight পরিবর্তন হওয়ায় মডেল পুরনো জ্ঞান ভুলে যায় — এটিকে Catastrophic Forgetting বলে। LoRA-এ মূল Weight Frozen রেখে শুধু ছোট Adapter ট্রেন করায়, মডেলের পুরনো জ্ঞান সম্পূর্ণ অক্ষুণ্ণ থাকে এবং নতুন জ্ঞান শুধু Adapter-এ যোগ হয়।

সারসংক্ষেপ ও মূল বিষয়গুলো

LoRA-এর মূল বিষয়গুলো একনজরে

বিষয়	LoRA কী করে
মূল Concept	Matrix Decomposition দিয়ে বড় Weight Update-কে ছোট দুটি Matrix-এ ভাঙে
Training Parameter	মাত্র A ও B Matrix (বাকি সব Frozen)
মূল Equation	$W_{\text{new}} = W_0 + B \times A$
Rank (r)	ছোট সংখ্যা (4, 8, 16, 32) — বড় হলে বেশি Parameter শেখে
Target Modules	সাধারণত q_proj ও v_proj (Attention Layer)
Inference	Weight Merge করে একটি Matrix-এ, কোনো Extra Latency নেই
সুবিধা	99%+ Parameter সাশ্রয়, দ্রুত Training, Catastrophic Forgetting নেই

QLoRA-এর মূল বিষয়গুলো একনজরে

বিষয়	QLoRA কী করে
মূল Concept	LoRA + Quantization — Base Model 4-bit-এ
Quantization Type	NF4 (4-bit Normal Float)

Double Quantization	Quantization Constants-ও Quantize হয়
Memory Saving	65B Model: 780GB → 48GB VRAM
Training Precision	LoRA Adapter 16-bit-এ ট্রেন হয়
বাস্তবায়ন	bitsandbytes library + PEFT library

মনে রাখার মূল বিষয়

LoRA: বড় ম্যাট্রিক্স (ΔW) কে দুটি ছোট ম্যাট্রিক্স ($B \times A$) দিয়ে approximate করা হয়। শুধু এই দুটি ছোট ম্যাট্রিক্স ট্রেন হয়। Inference-এ merge করে একটি matrix হয়।

QLoRA: LoRA-এর উপরে Quantization যোগ করে Base Model-কে 4-bit-এ রেখে আরও কম মেমরিতে কাজ করা সম্ভব হয়।

Practical নিয়ম: সাধারণ task-এ Rank 8 ব্যবহার করো (Microsoft-এর recommendation)। জটিল task-এ Rank 16-64 ব্যবহার করো।

চূড়ান্ত উপমা: লাইব্রেরি ও নোটবুক

মেইন মডেল হলো বিশাল একটি লাইব্রেরি (Frozen)। LoRA হলো তোমার নোটবুক — পড়ার সময় বইয়ে দাগ না দিয়ে নোটবুকে লিখে রাখছ। উত্তর দেওয়ার সময় বই ও নোটবুক মিলিয়ে উত্তর দিচ্ছ। QLoRA হলো সেই লাইব্রেরির বইগুলোকে ছোট করে compress করে রাখা (4-bit) — জায়গা কম লাগে, কিন্তু পড়া যায়।

LoRA ও QLoRA — সম্পূর্ণ বাংলা গাইড

Parameter Efficient Fine-Tuning · Low-Rank Adaptation · Quantized LoRA

Transformer Architecture · LLM Fine-Tuning · Bengali ML Guide